Name: Vanshika Wadhwani
Class: D20A
Roll no: 35

# Experiment:5

**Aim:**

To study the structure of Solidity programs, basic programming constructs, and the environment used for compilation and deployment of smart contracts.

**Theory:**

Solidity is a statically typed, contract-oriented programming language used to develop smart contracts on Ethereum and other EVM-compatible blockchains. Smart contracts are programs stored on a blockchain that automatically execute instructions according to predefined rules.

Remix IDE is an online development platform used to write, compile, deploy, and test Solidity smart contracts. It provides a Remix VM, which simulates a blockchain environment and provides test accounts with Ether for performing transactions.

A Solidity program generally contains a license identifier, a compiler version declaration using pragma, and one or more contract definitions.

**1. Primitive Data Types, Variables, and Functions**

Data Types: Solidity supports several basic data types, such as uint for unsigned integers, int for signed integers, bool for true or false values, address for blockchain addresses, bytes for binary data, and string for text. Variables that are not explicitly initialized receive default values.

Variables: Solidity variables are classified into three main categories:

- State variables: Declared outside functions and stored permanently on the blockchain.
- Local variables: Declared inside functions and used temporarily during execution.
- Global variables: Provide information about the current transaction or blockchain, such as msg.sender, msg.value, block.timestamp, and block.number.

Functions: Functions are used to perform operations on contract data. State-changing functions generally require transactions and consume gas. A view function reads stored data without

modifying it, whereas a pure function neither reads nor modifies blockchain state. External calls to these read-only functions can usually be performed without transaction fees.

### 2. Inputs and Outputs of Functions

Functions can accept one or more input parameters and return one or more output values. Solidity supports multiple return values, including tuples, and allows return variables to be named.

Arrays can be passed as inputs or returned as outputs, but handling large arrays may increase gas consumption. Mappings cannot be directly passed as parameters or returned by public or external functions.

### 3. Visibility, Modifiers, Constructors, and Inheritance

#### Visibility

Visibility determines where a function or variable can be accessed.

| Visibility | Description |
|---|---|
| public | Accessible internally and externally, including through derived contracts where applicable. |
| private | Accessible only within the contract in which it is declared. |
| internal | Accessible within the contract and its derived contracts. |
| external | Intended to be called from outside the contract; internal calls generally require an external-style call. |

#### Modifiers

Modifiers are reusable code blocks that control how functions execute. They are commonly used to restrict access, validate conditions, and improve security. The _ ; statement indicates where the modified function's code is executed.

**Constructors**

A constructor executes only once when a contract is deployed. It is commonly used to initialize variables or assign the deployer's address as the contract owner.

**Inheritance**

Inheritance allows one contract to reuse the functions and variables of another contract. It is implemented using the is keyword, such as contract Child is Base.

**4. Control Flow: Conditional Statements and Loops**

Control-flow statements determine the order in which program instructions execute.

- Conditional statements: if, else if, and else execute code according to specified conditions.
- Ternary operator: The expression condition ? a : b selects one of two values based on a condition.
- Loops: for, while, and do-while repeatedly execute a block of code.
- Break: Immediately terminates a loop.
- Continue: Skips the current iteration and proceeds to the next one.

Loops must be designed carefully in smart contracts because excessive iterations consume gas and may cause transactions to fail.

**5. Data Structures: Arrays, Mappings, Structures, and Enumerations**

Arrays: Arrays store multiple values of the same type. They can have a fixed size, such as uint[3], or a dynamic size, such as uint[]. Common operations include push(), pop(), length, and delete, depending on the array type.

Mappings: Mappings store key-value pairs and allow efficient access to values using their keys. They do not directly support iteration or provide a length property. Unassigned keys return the default value of the mapped type.

Structures (Structs): Structures group related variables of different types into a single custom data type. For example, a Todo structure may contain a task description and a completion status.

Enumerations (Enums): Enums define a set of named constant values. For example, enum Size { S, M, L } represents three size options. The first member is the default value, and enums are represented internally using an integer type.

### 6. Data Locations

Data locations specify where data is stored and how it is accessed.

| Data Location | Description |
|---|---|
| storage | Refers to persistent contract data stored on the blockchain. |
| memory | Holds temporary data during function execution. |
| calldata | Holds read-only function input data, particularly for external functions. |

Assigning a storage value to memory creates a copy of the data. A storage reference, however, refers to the original data, so modifications can affect the contract's state. Storage reference parameters are permitted only in internal and private functions.

### 7. Transactions: Ether, Wei, Gas, and Sending Ether

#### Units of Ether

Ether is the cryptocurrency used on the Ethereum network. Its smaller units are:

- 1 wei = the smallest unit of Ether.
- 1 gwei = $10^9$ wei.
- 1 ether = $10^{18}$ wei.

#### Gas and Gas Price

Gas measures the computational effort required to execute operations on a blockchain. The transaction fee is calculated as:

Transaction Fee = Gas Used × Gas Price

If a transaction runs out of gas, its state changes are reverted, but the gas already consumed is not refunded.

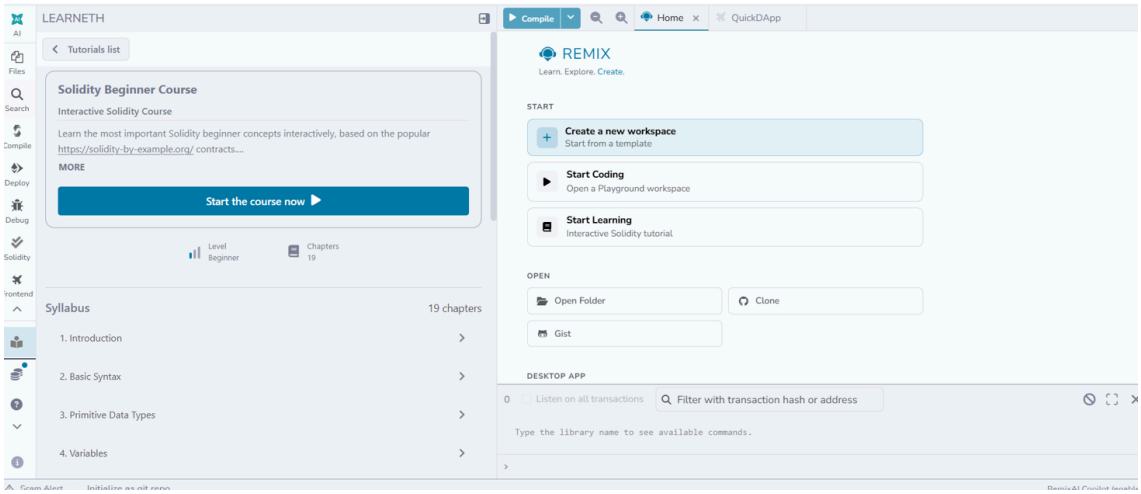
**Sending and Receiving Ether**

A function must be marked payable to receive Ether directly through a transaction.

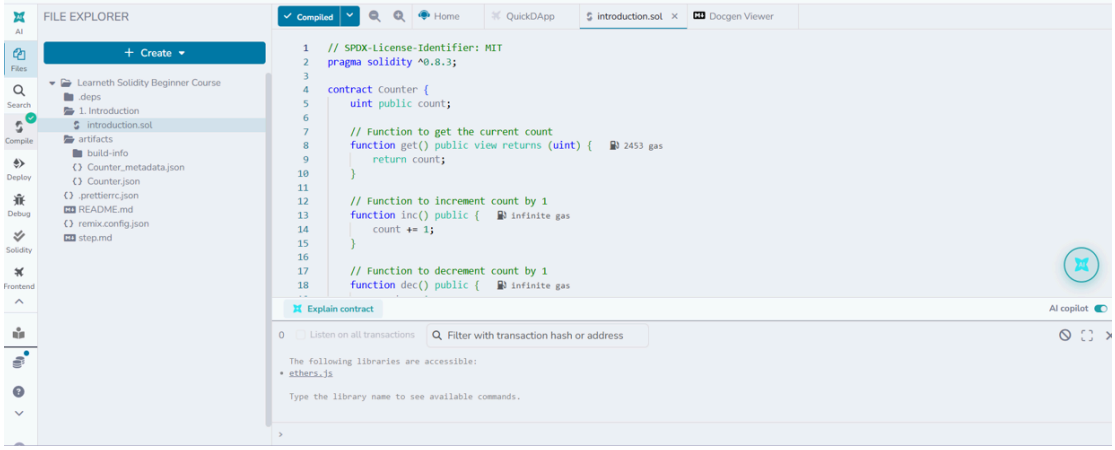
Solidity provides three commonly discussed methods for transferring Ether:

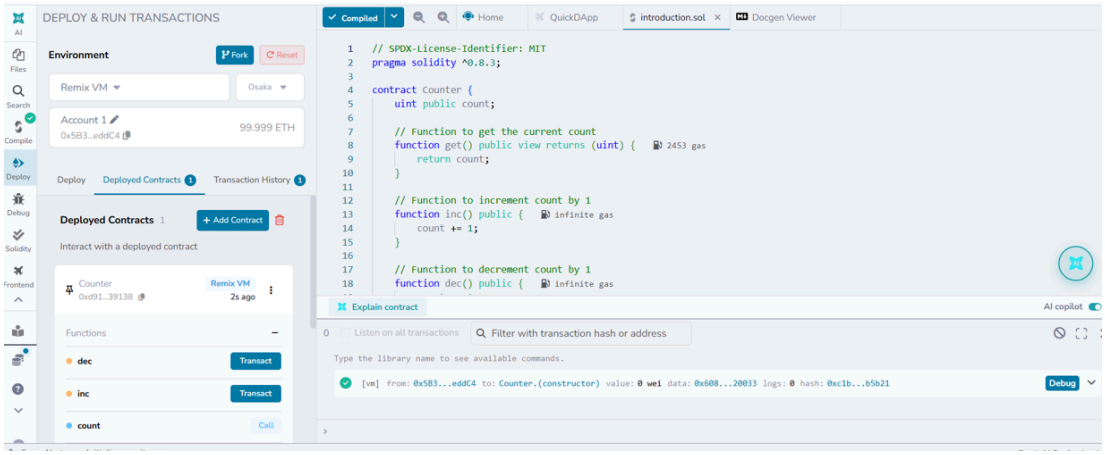
- transfer(): Reverts the transfer if it fails and forwards a limited amount of gas.
- send(): Returns a Boolean value indicating success or failure and forwards a limited amount of gas.
- call{value: amount}(""): Transfers Ether using a low-level call and returns a success status. It is generally preferred over transfer() and send() in modern Solidity, but it must be used carefully with appropriate error handling and reentrancy protection.

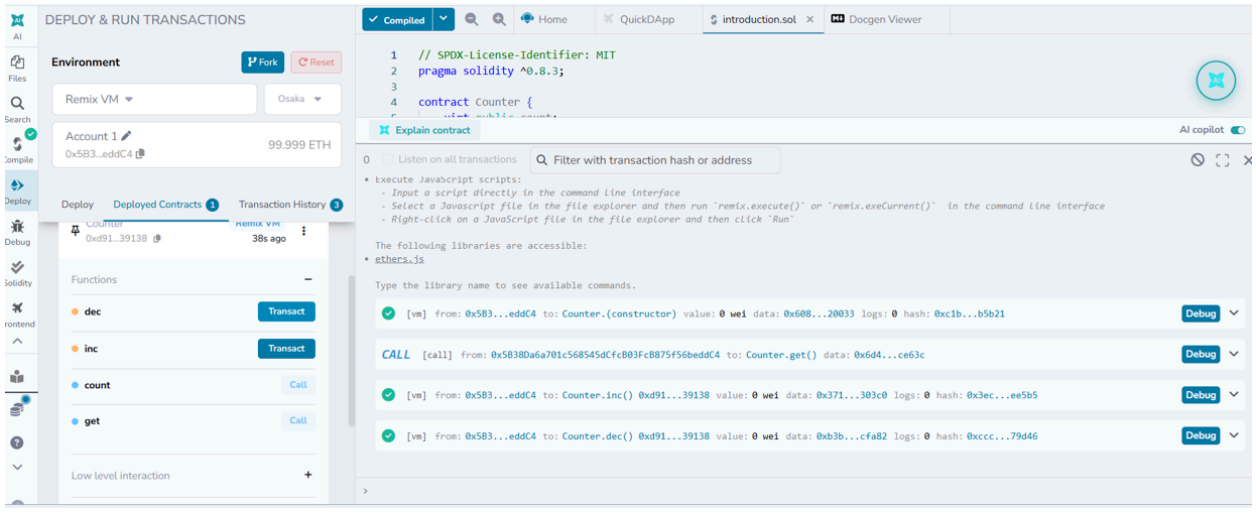
Output:



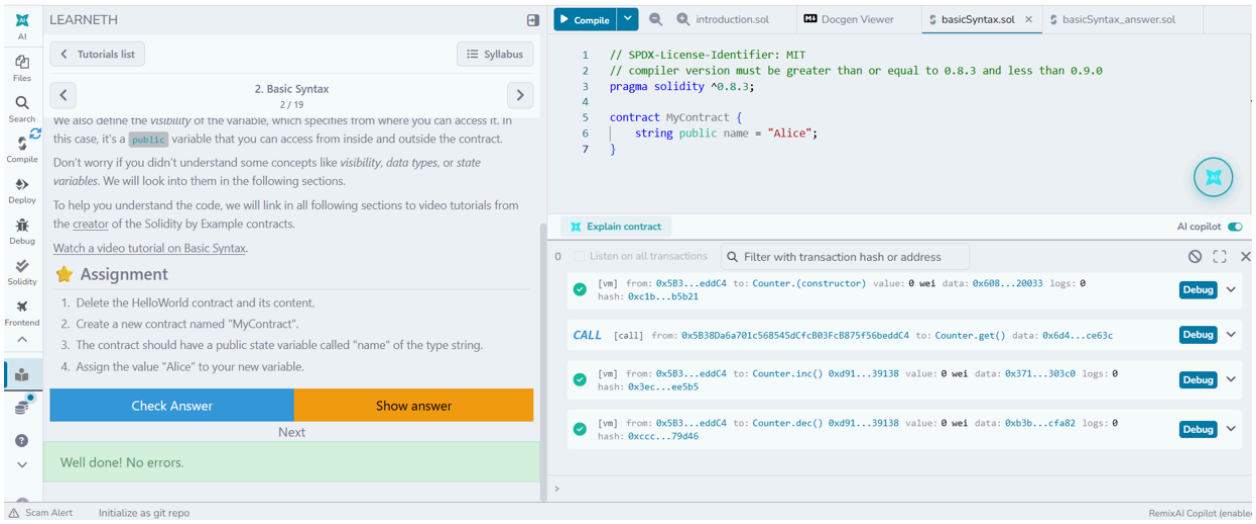

Tutorial:1


FILE EXPLORER
Create
Learneth Solidity Beginner Course
.deps
1_Introduction
introduction.sol
artifacts
build-info
Counter_metadata.json
Counter.json
.prettierrc.json
README.md
remix.config.json
step.md
Compiled
Home
QuickDApp
introduction.sol
Docgen Viewer
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;
contract Counter {
uint public count;
// Function to get the current count
function get() public view returns (uint) {
2453 gas
return count;
}
// Function to increment count by 1
function inc() public {
infinite gas
count += 1;
}
// Function to decrement count by 1
function dec() public {
infinite gas
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or address
The following libraries are accessible:
ethers.js
Type the library name to see available commands.

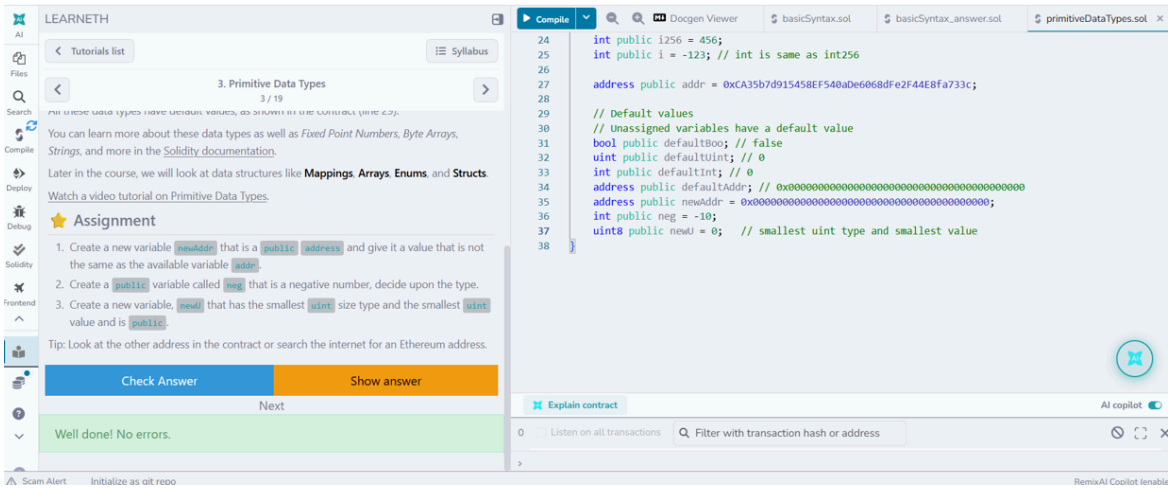

DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Deploy
Deployed Contracts 1
Transaction History 1
Add Contract
Interact with a deployed contract
Counter
0xd91...39138
Remix VM
2s ago
Functions
dec
Transact
inc
Transact
count
Call
Compiled
Home
QuickDApp
introduction.sol
Docgen Viewer
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;
contract Counter {
uint public count;
// Function to get the current count
function get() public view returns (uint) {
2453 gas
return count;
}
// Function to increment count by 1
function inc() public {
infinite gas
count += 1;
}
// Function to decrement count by 1
function dec() public {
infinite gas
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or address
Type the library name to see available commands.
[vm] from: 0x5B3...eddC4 to: Counter.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xc1b...b5b21
Debug


DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Deploy
Deployed Contracts 1
Transaction History 3
Counter
0xd91...39138
Remix VM
38s ago
Functions
dec
Transact
inc
Transact
count
Call
get
Call
Low level interaction
Compiled
Home
QuickDApp
introduction.sol
Docgen Viewer
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;
contract Counter {
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or address
Execute JavaScript scripts:
Input a script directly in the command line interface
Select a Javascript file in the file explorer and then run `remix.execute()` or `remix.exeCurrent()` in the command line interface
Right-click on a JavaScript file in the file explorer and then click `Run`
The following libraries are accessible:
ethers.js
Type the library name to see available commands.
[vm] from: 0x5B3...eddC4 to: Counter.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xc1b...b5b21
Debug
CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get() data: 0x6d4...ce63c
Debug
[vm] from: 0x5B3...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0x3ec...ee5b5
Debug
[vm] from: 0x5B3...eddC4 to: Counter.dec() 0xd91...39138 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0xccc...79d46
Debug

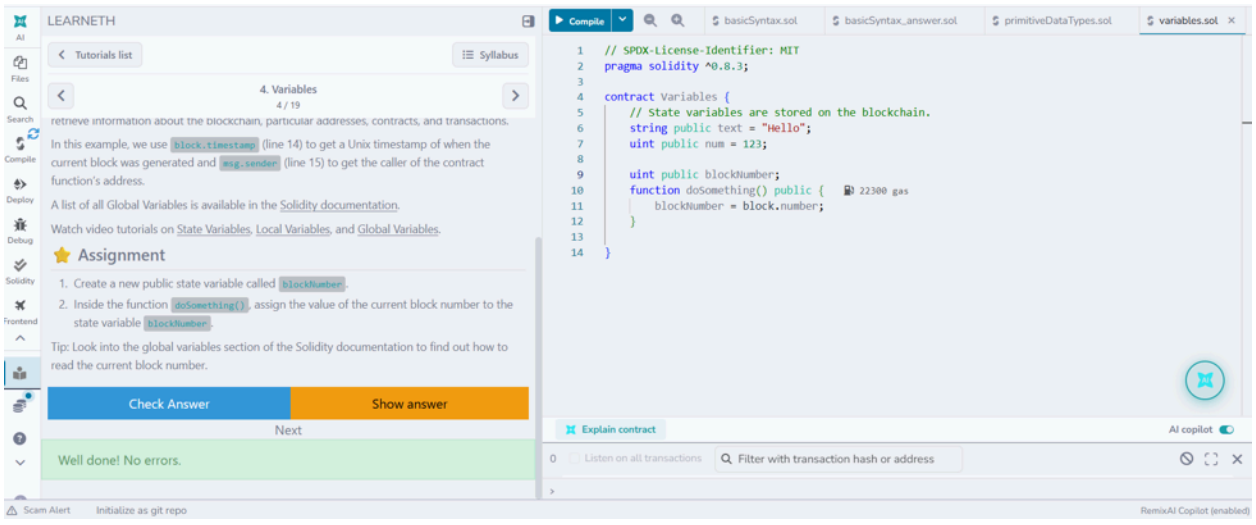
Tutorial:2



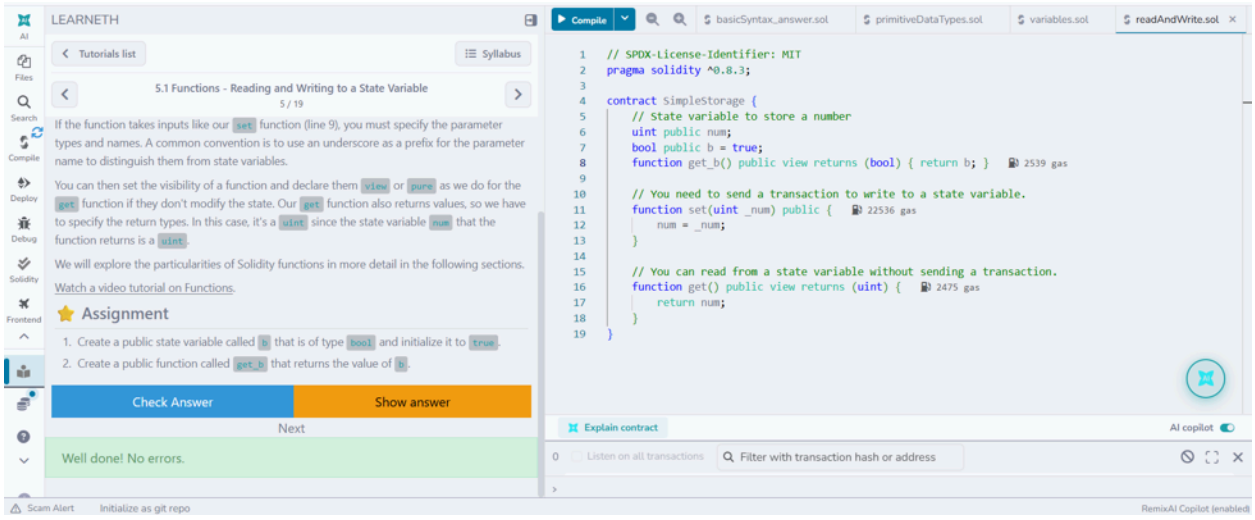

## Tutorial:3



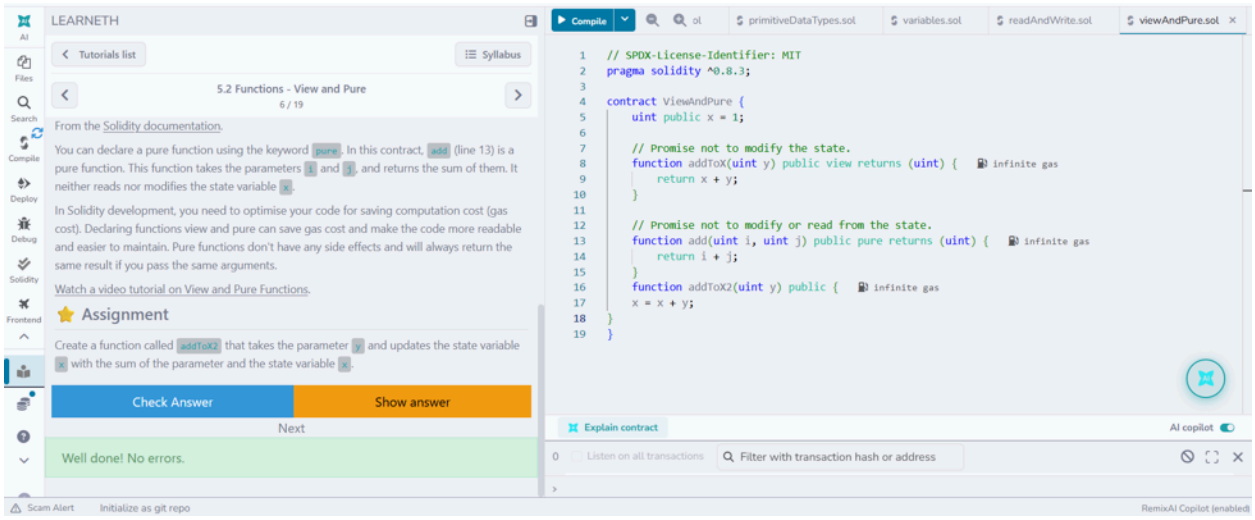

## Tutorial:4



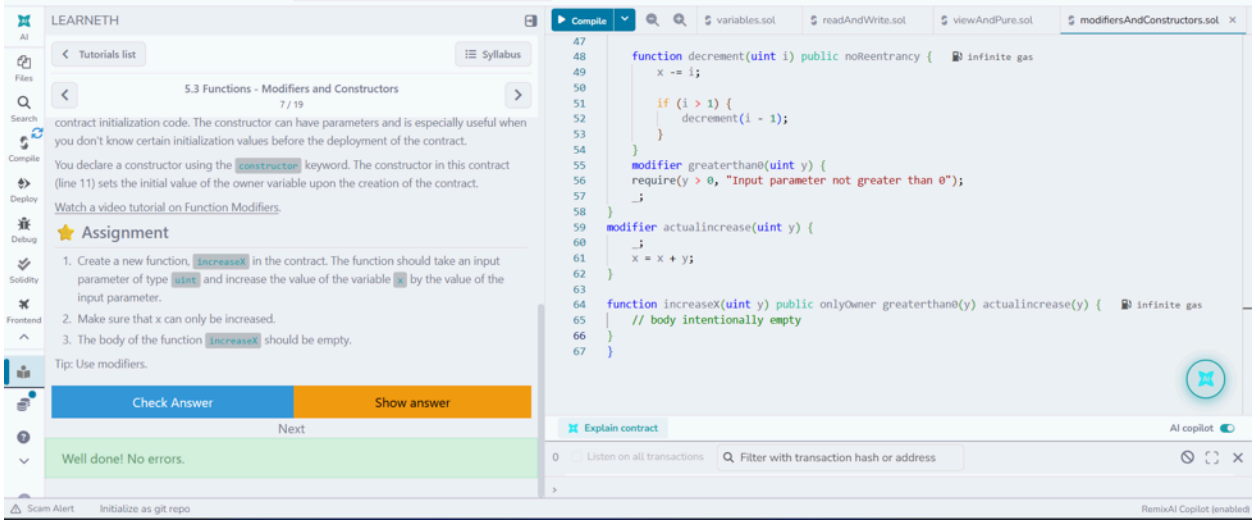
Tutorial:5



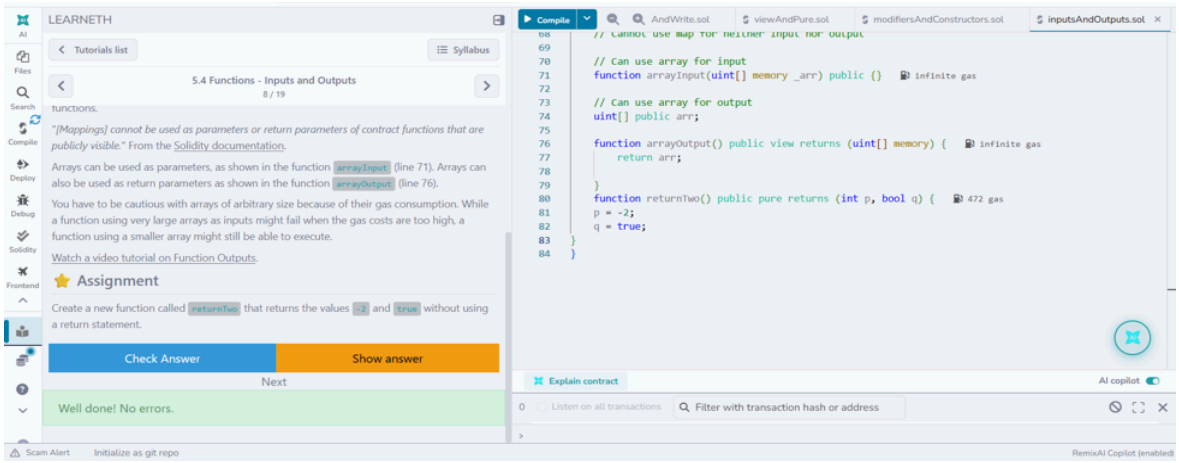

Tutorial:6



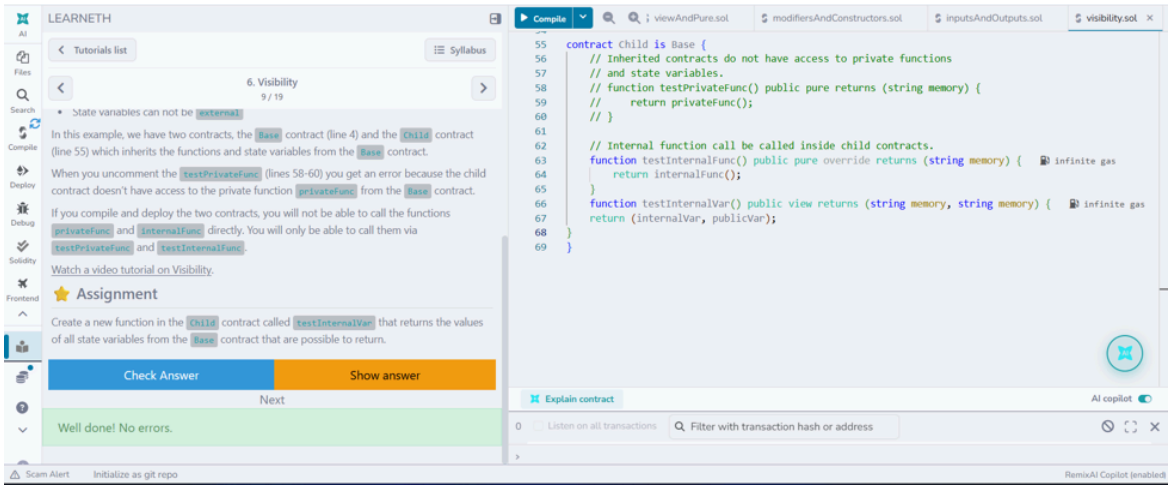

Tutorial:7



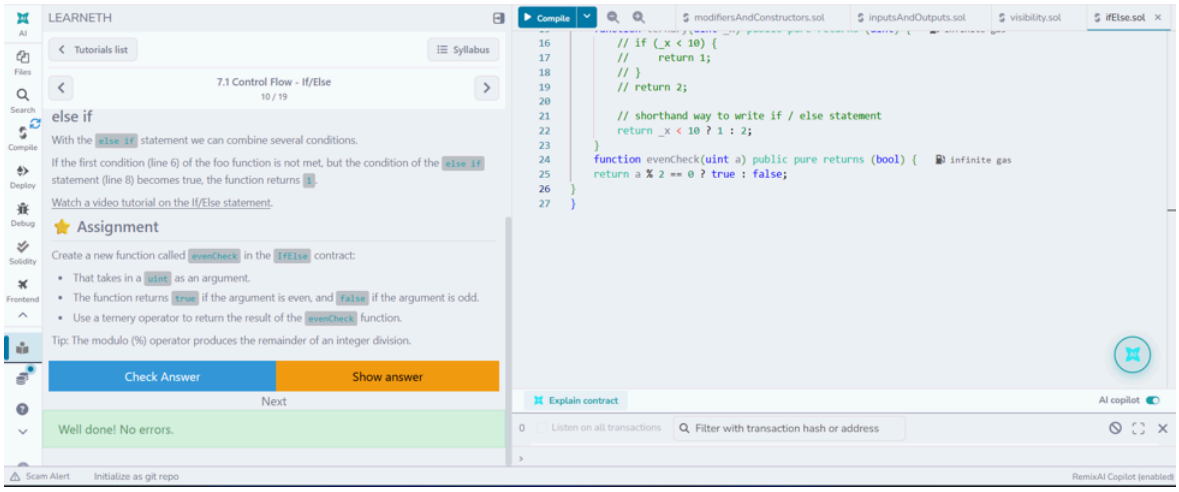

Tutorial:8



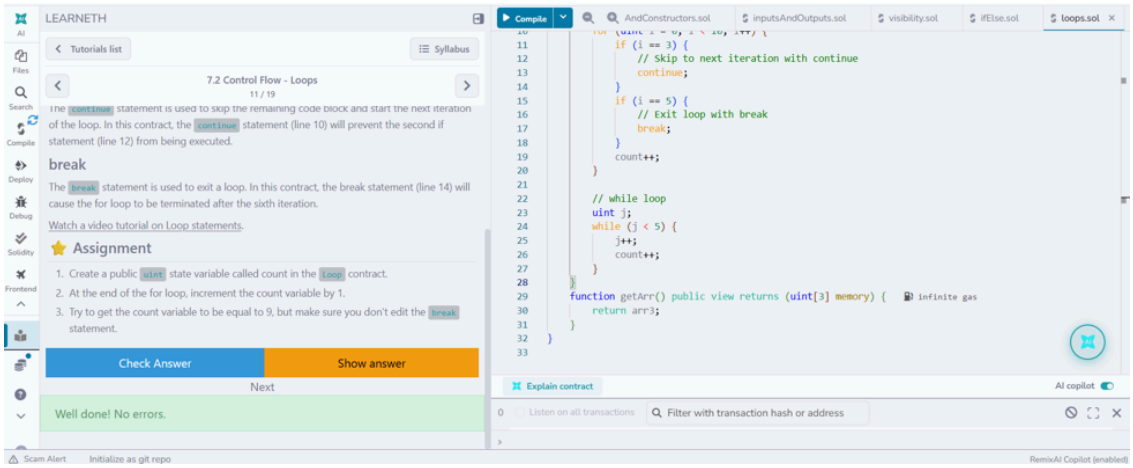

Tutorial:9



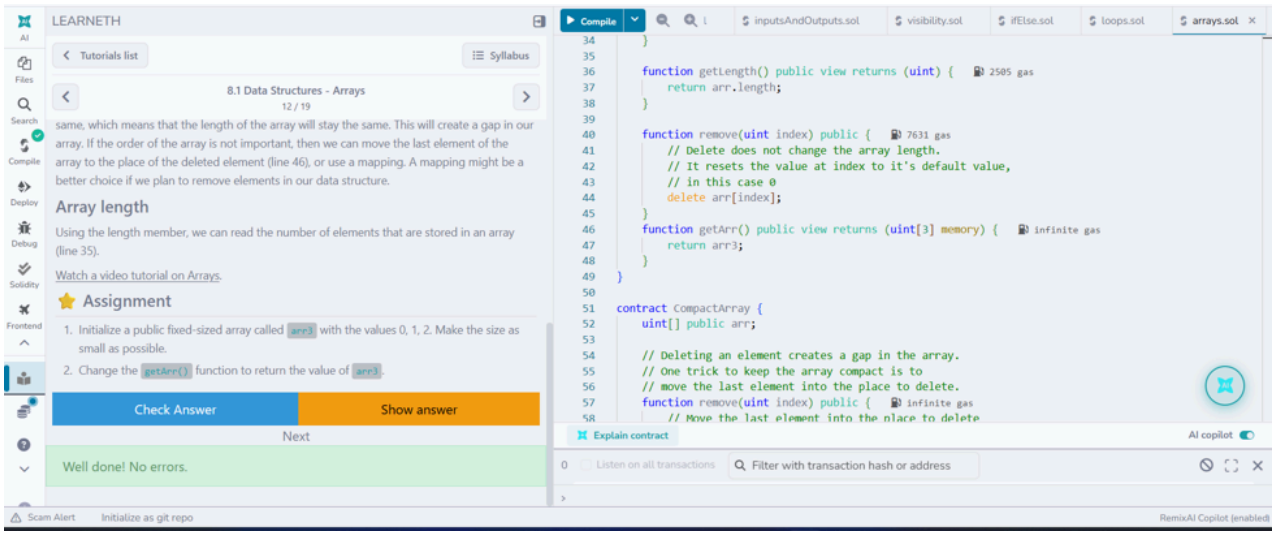
Tutorial:10



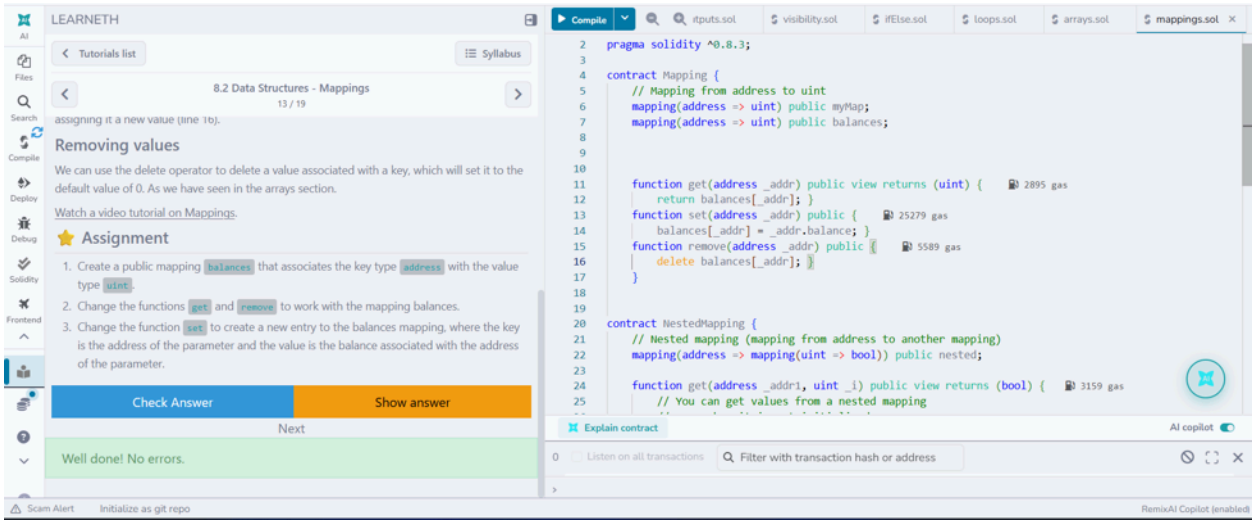

Tutorial:11



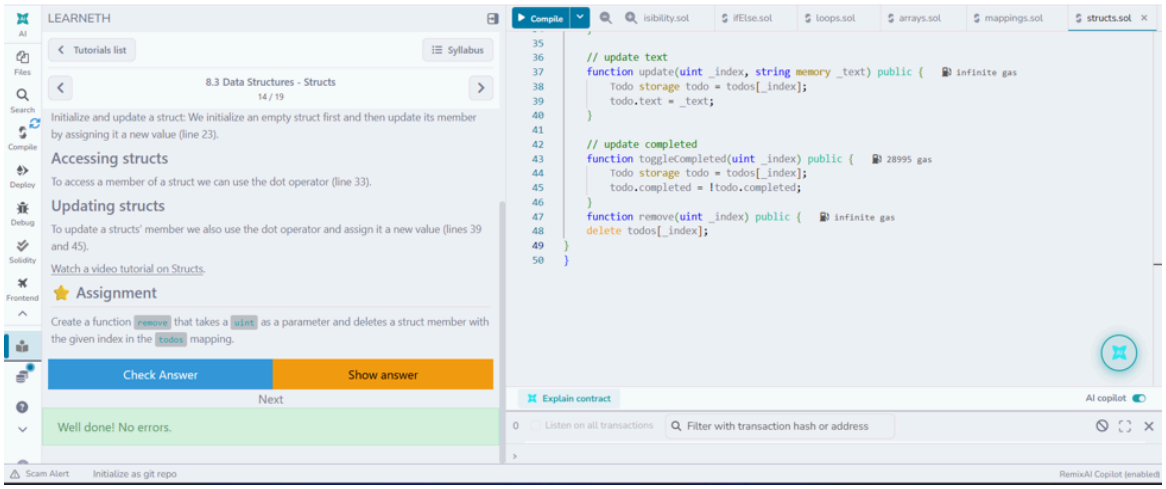

Tutorial: 12



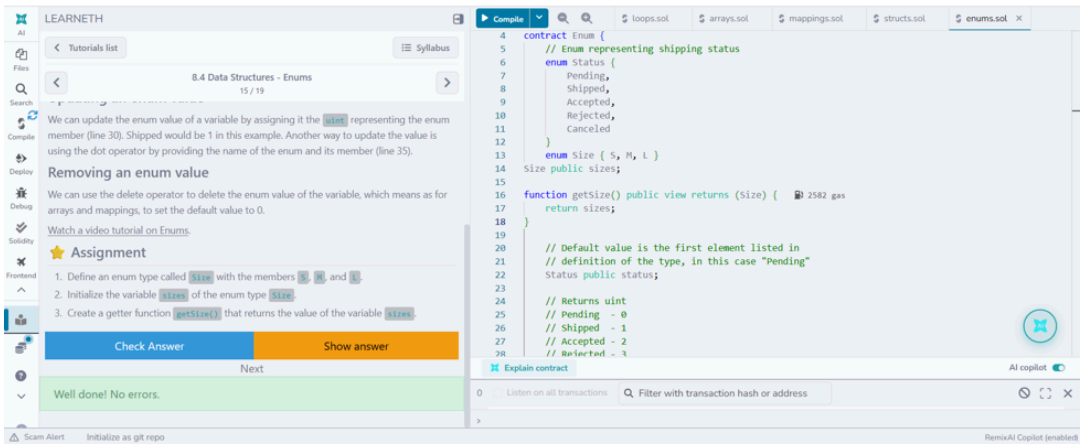
Tutorial:13



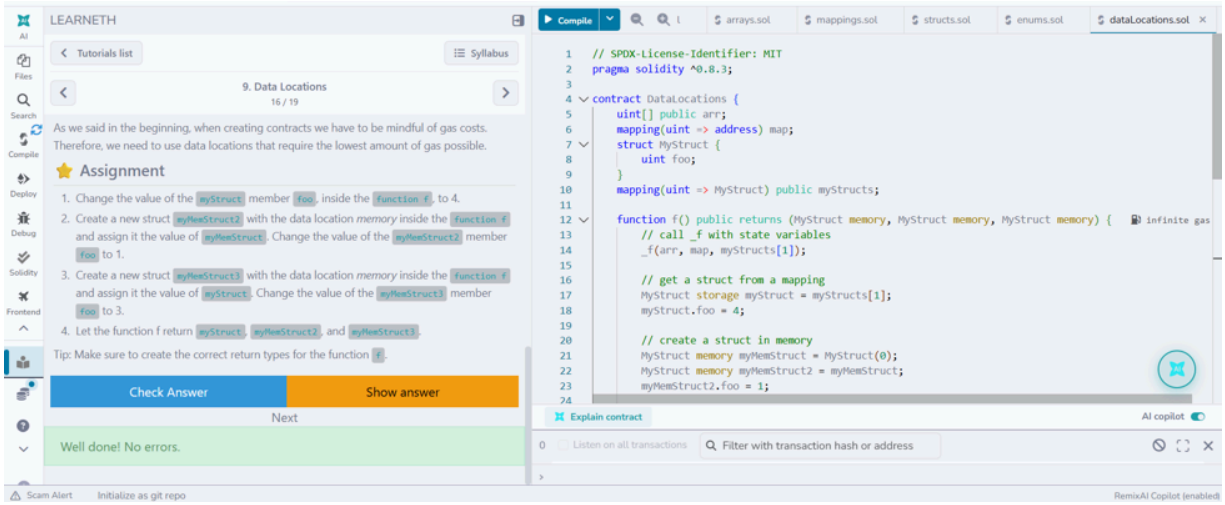

Tutorial:14



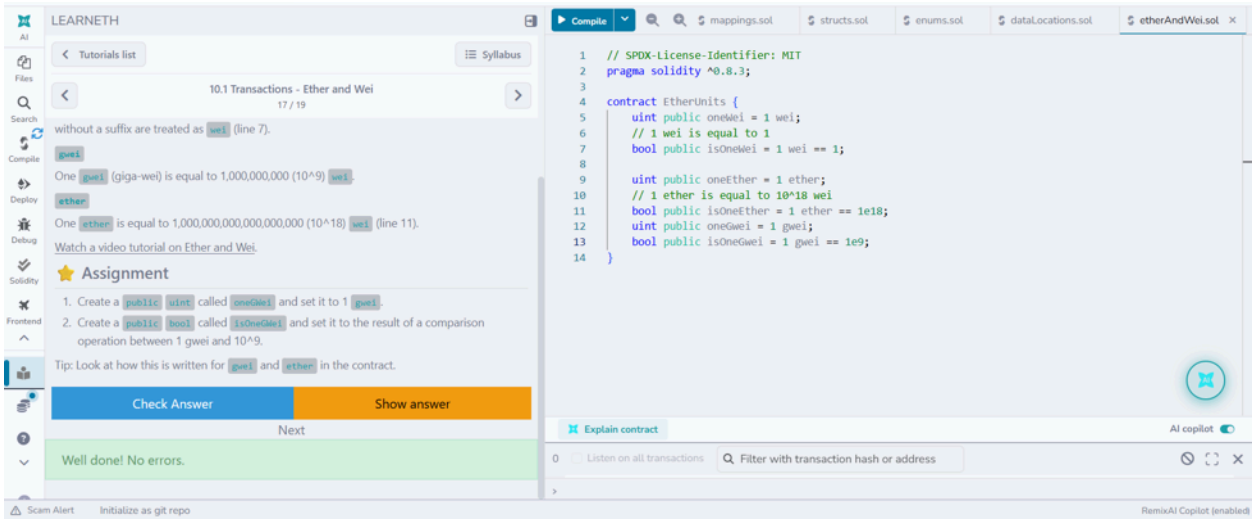

Tutorial:15



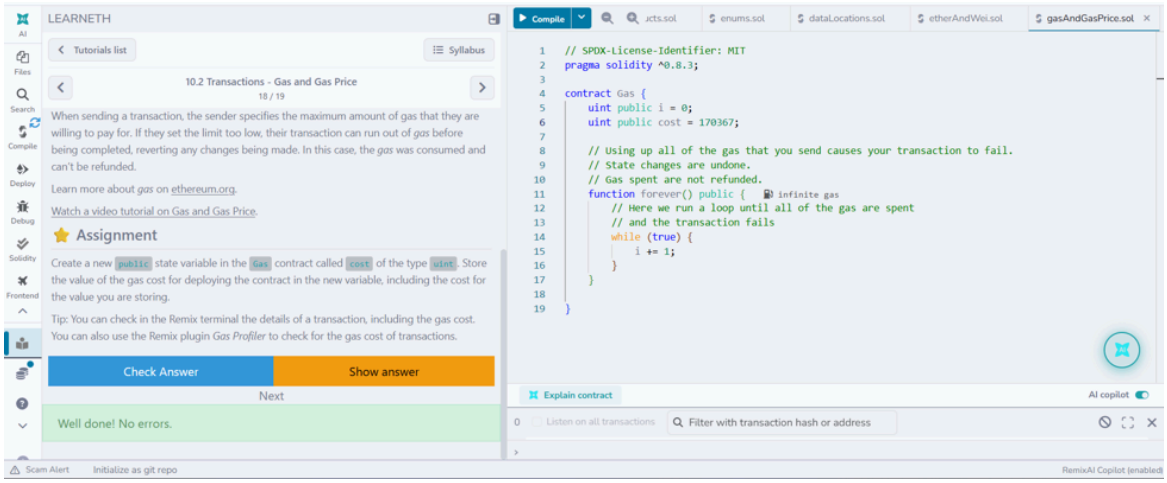
Tutorial:16



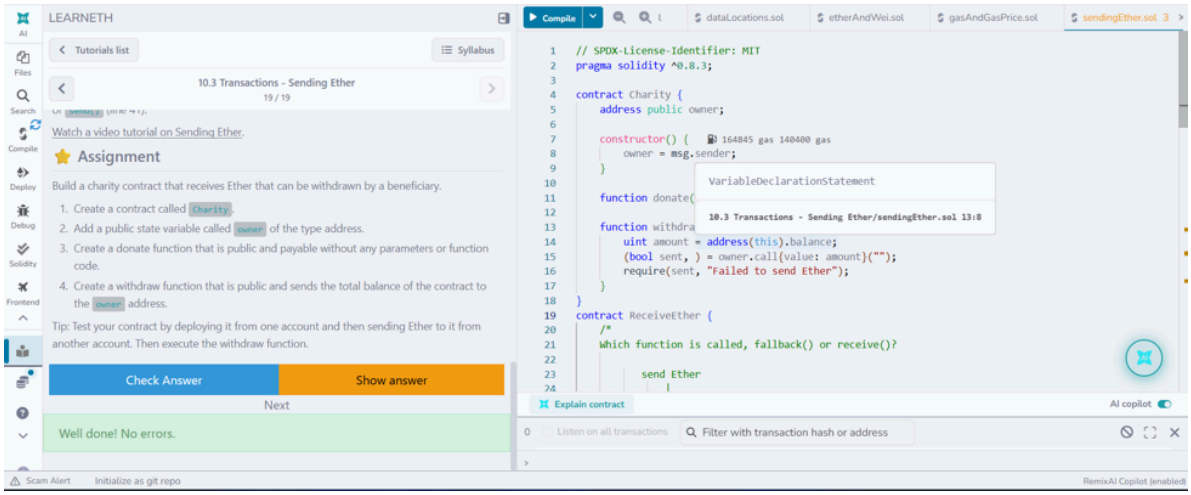

Tutorial:17




Tutorial:18



Tutorial:19




## Conclusion

Thus, the structure of Solidity programs, fundamental data types, variables, functions, control-flow statements, data structures, data locations, and transaction concepts were studied. The role of Remix IDE in compiling, deploying, and testing smart contracts was also understood. These concepts provide the foundation for developing secure and efficient blockchain applications.